

M2 Data Science – M2 Intelligence Artificielle

Data Mining

Miner du texte

(et autres données non structurées)



Définitions: données textuelles

- Dans le vrai monde de la réalité véritable, énormément de données sont dites **non structurées**
 - Elles ne sont pas sous forme de colonnes bien rangées (données structurées)
- Parmi ces données non structurées, les **données textuelles** sont sûrement les plus omniprésentes
 - Tweets et descriptions
 - Avis et Commentaires
 - Rapports médicaux
 - Etc.
- Nécessite des traitements particulier pour extraire des informations de ce type de données
 - Ex. Text Mining
- On parle de jeux de données **semi-structurés**, dans les cas où un jeu de données contient une plusieurs variables non structurées

organisation

HCL HÔPITAL ED-OUARD HERRIOT

SERVICE MEDICAL HOSPITALIER CENTRE
3, PLACE D'ALEXANDRE
69631 LYON CEDEX 07 FRANCE

LYON le DD/MM/AAAA

lieu date

«Publipostage_AdresseDestinataire»

«Publipostage_CopiePrincipal» médecin
«Publipostage_CopieAutre»

COMPTE RENDU DE CONSULTATION

cher confrère,

patient

symptômes

traitement

Exemple factice de compte-rendu de consultation

Miner des données textuelles

Place à la pratique !

- Sur [kaggle](#), téléchargez l'un des jeux de données suivants:
 1. [Bitcoin Tweets](#)
 2. [Rick&Morty Scripts](#)
 3. [Social Media Influence \(by text mining\)](#)
 4. [Superheroes NLP Dataset](#)
 5. [Emotions](#)
 6. [Taylor Swift Song Lyrics from all the albums](#)
 7. [Google Play App Reviews Dataset](#)
- Vérifiez et nettoyez vos données !
 - Attention à l'encodage des textes
- Discutez des informations que vous pouvez extraire de ces données et de comment les extraire
- Faites des tests et discutez de vos résultats

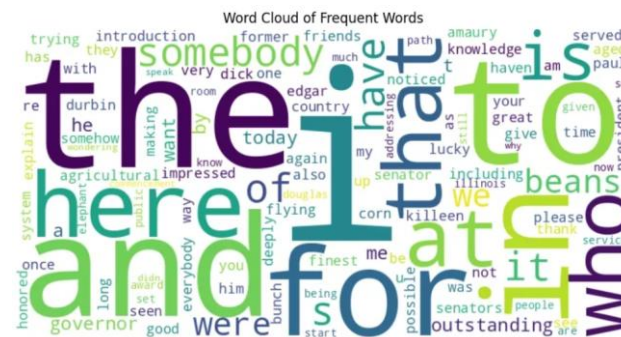
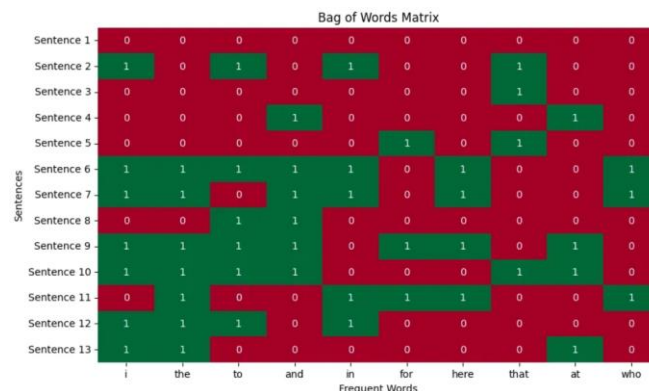


4

1. On crée un **dictionnaire** à partir de l'ensemble des mots présents dans un corpus de n textes
2. Pour chaque mot, on calcule sa fréquence d'occurrence dans le corpus de textes
3. On garde les m mots les plus fréquents
4. On crée une matrice de taille $n \times m$
5. Pour chaque texte du corpus
 - Pour chaque mot du dictionnaire
 - Si le mot est présent dans le texte: la cellule correspondante dans la matrice est mise à 1, sinon 0
6. À partir de cette matrice, il est possible de créer un nuage de mots

- Les mots de liaisons (de, en, le, la, les, etc.) sont souvent les plus fréquents, de même pour la ponctuation
- Sensible à la casse
- Ne prends pas en compte le contexte de chaque mot

Words	Frequency
i	12
the	9
to	8
and	7
in	5
for	5
here	5
that	4
ok	4
who	4



Préparer les données textuelles

Pre-processing « simple »

Bonnes pratiques:

- Mettre tout en minuscule
- Supprimer les ponctuations
- Supprimer les espaces redondants
- Supprimer les mots de liaisons, ou « *stopwords* »
 - Des listes existent (ex. stopwords de la librairie nltk.corpus)
 - Il peut parfois être nécessaire de définir une liste de *stopwords* spécifique à un domaine

Stop Words

- a
- of
- on
- I
- for
- with
- the
- at
- from
- in
- to

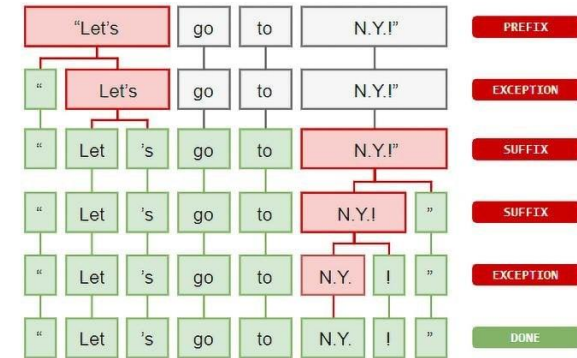
[“**This**”, “**is**”, “**a**”, “**test**”]

✓ X X ✓

Préparer les données textuelles

Tokenisation, stemming, et lemmatisation

- **Tokenisation:** Découper un texte en sous-éléments
 - Phrases, mots, « tokens », lettres, etc.
 - Créer un **vocabulaire** à partir du corpus de textes
- **Stemming:** Ramener mots à sa racine, son radical
 - Aide à normaliser les textes et faciliter leur analyse
 - Créé un « stemmed » corpus
- **Lemmatisation:** Convertit un mot à une base signifiante
 - Utilise un dictionnaire de conversion appelé **Lemme** (ou *lemma*), par exemple: WordNet
 - À l'inverse du stemming, la lemmatisation s'assure que chaque mot soit ramené à un mot réel existant
 - Limite: par défaut, considère chaque mot comme un nom



Stemming

adjustable → adjust
 formality → formalit
 formaliti → formal
 airliner → airlin

Stemming

change
 changing
 changes
 changed
 changer

→ change

Lemmatization

was → (to) be
 better → good
 meeting → meeting

Lemmatization

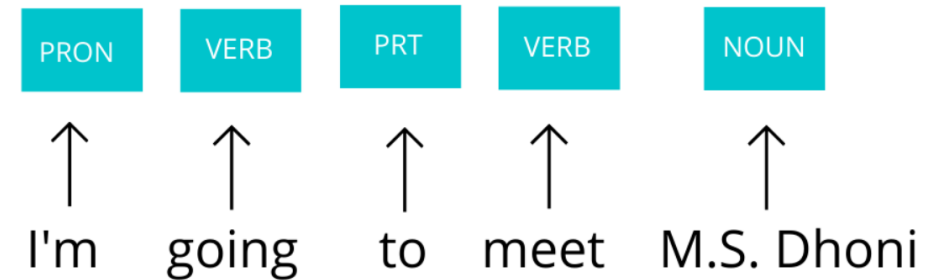
change
 changing
 changes
 changed
 changer

→ change

Préparer les données textuelles

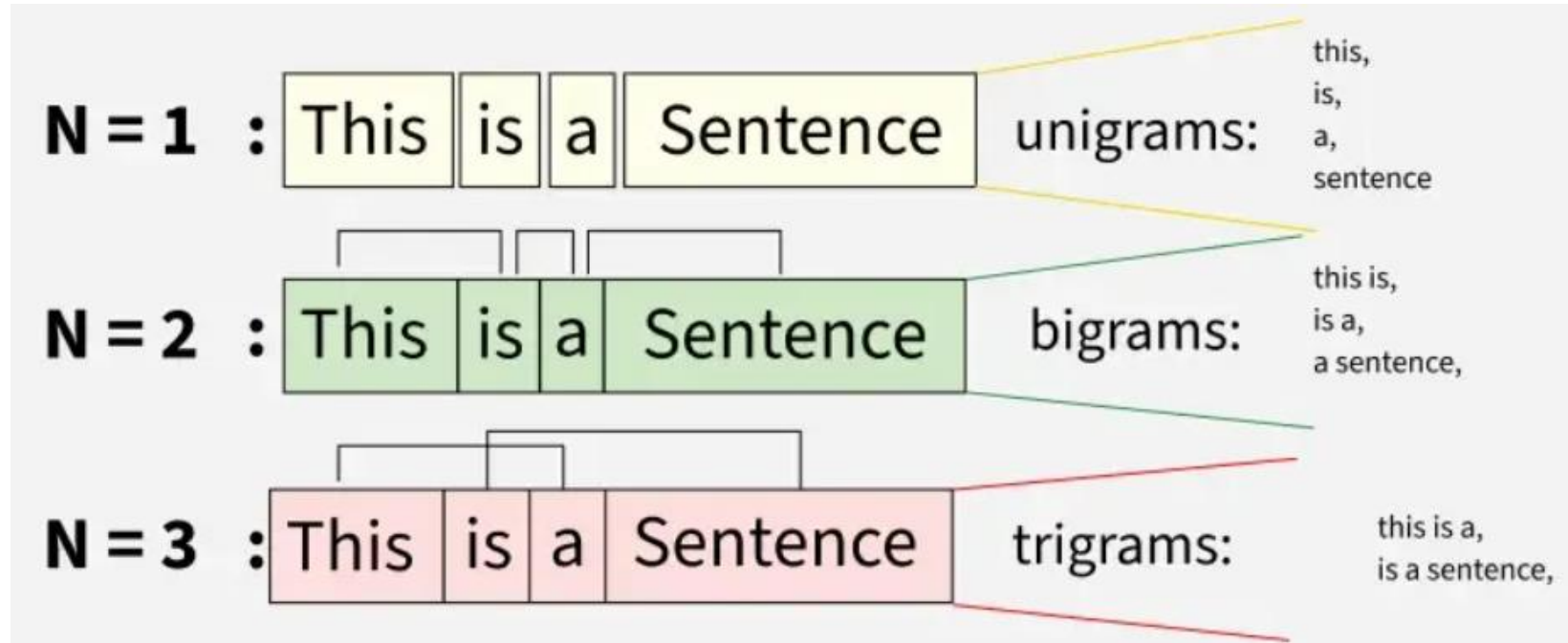
Part of Speech (PoS) Tagging

- **Objectif:** Identifier le rôle de chaque mot et les labeliser en conséquence
 - Un mot peut être: un nom, un pronom, un verbe, etc.
- Plusieurs méthodes possibles:
 - Basées sur des règles (Rule-based PoS Tagging)
 - Sur des transformations itératives liées au contexte (Transformation-based PoS Tagging)
 - Basées sur des statistiques
 - Hidden Markov Models (HMMs)
 - Conditional Random Fields (CRFs)
- Améliore la lemmatisation en indiquant le type de dictionnaire à utiliser



Données textuelles

N-grams



⇒ Permet d'analyser des mots dans leurs contextes

⇒ Ex. « Un homme **poli** » et « un meuble **poli** », même mot, mais un sens différent

⇒ Les calculs de fréquences s'appliquent de la manière

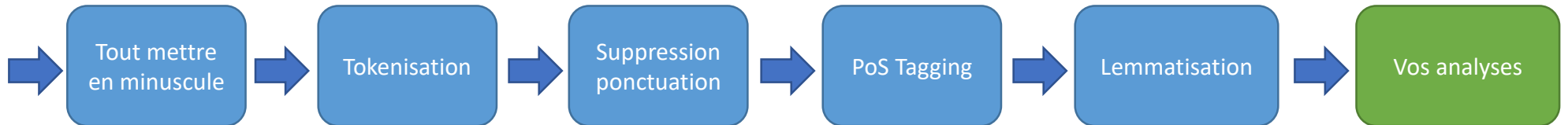
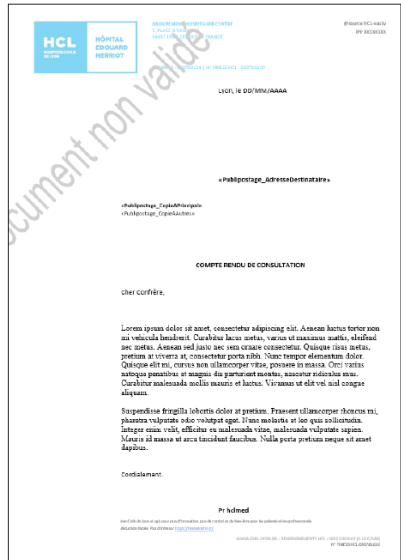
⇒ Un groupe de mots fréquent peut être plus significatif que plusieurs mots fréquents

⇒ Penser à tester plusieurs tailles de n-gram

Préparer les données textuelles

Synthèse et bonnes pratiques

Texte brut



Miner des données textuelles

Place à la pratique !

- Reprenez les jeux de données téléchargés précédemment:
 1. [Bitcoin Tweets](#)
 2. [Rick&Morty Scripts](#)
 3. [Social Media Influence \(by text mining\)](#)
 4. [Superheroes NLP Dataset](#)
 5. [Emotions](#)
 6. [Taylor Swift Song Lyrics from all the albums](#)
 7. [Google Play App Reviews Dataset](#)
- Tester les notions que l'on vient de voir
- Discutez sur comment caractériser/résumer un texte et regrouper les textes qui se ressemblent
- Faites des tests et discutez de vos résultats



Données textuelles

Term Frequency – Inverse Document Frequency (TF-IDF)

- **Objectif:** Déterminer les termes importants pour un document par rapport au reste des documents du corpus
 - Peut se faire aussi sur des n-grams de différentes tailles
 - Permet de mettre en évidence des mots (ou des phrases) clés caractérisant (ou résumant) un document
- Soit D le corpus de documents étudié et T l'ensemble des termes utilisés dans ce corpus, TF-IDF se base sur:
 - TF: la fréquence d'un terme $t \in T$ dans un document $d \in D$
 - $TF(t, d) = \frac{\text{Nombre d'occurrence du terme } t \text{ dans le document } d}{\text{Nombre total de termes dans le document } d}$
 - IDF: L'inverse de la fréquence de document $d \in D$ contenant ce même terme t :
 - $IDF(t, D) = \log\left(\frac{\text{Nombre total de documents dans } D}{\text{Nombre de documents contenant le terme } t}\right)$
 - $TF - IDF(t, d, D) = TF(t, d) \times IDF(t, D)$
- Un terme très fréquent dans un document, mais seulement dans celui-ci, est plus significatif qu'un terme très fréquent dans tous les documents
- Peut aussi être calculé pour un sous-groupe de documents

Données textuelles

Topic modeling – Latent Semantic Analysis (LSA)

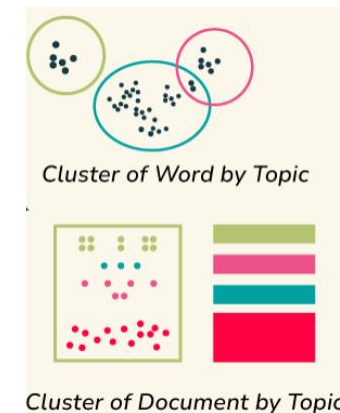
Algorithme:

1. Créer une matrice terme-document comptant le nombre d'occurrence de chaque terme dans chaque document
 - Similaire à créer un « bag of words »
 - TF-IDF peut être utilisé au lieu du nombre d'occurrence
2. Réduire le nombre de dimension
 - Par défaut, LSA utilise [Singular Value Decomposition \(SVD\)](#)
 - Mais d'autres méthodes sont possible
3. Regrouper les termes « proches » pour définir des « topics »
4. Classer les documents dans les topics trouvés

	D1	D2	D3
The	1	0	0
dog	1	0	0
ran	1	0	0
through	1	0	0
the	2	1	0
park	1	1	1
and	0	2	1
chased	0	1	0
a	0	1	0
ball	0	0	1
into	0	0	1
the	0	0	1
bushes	0	0	1



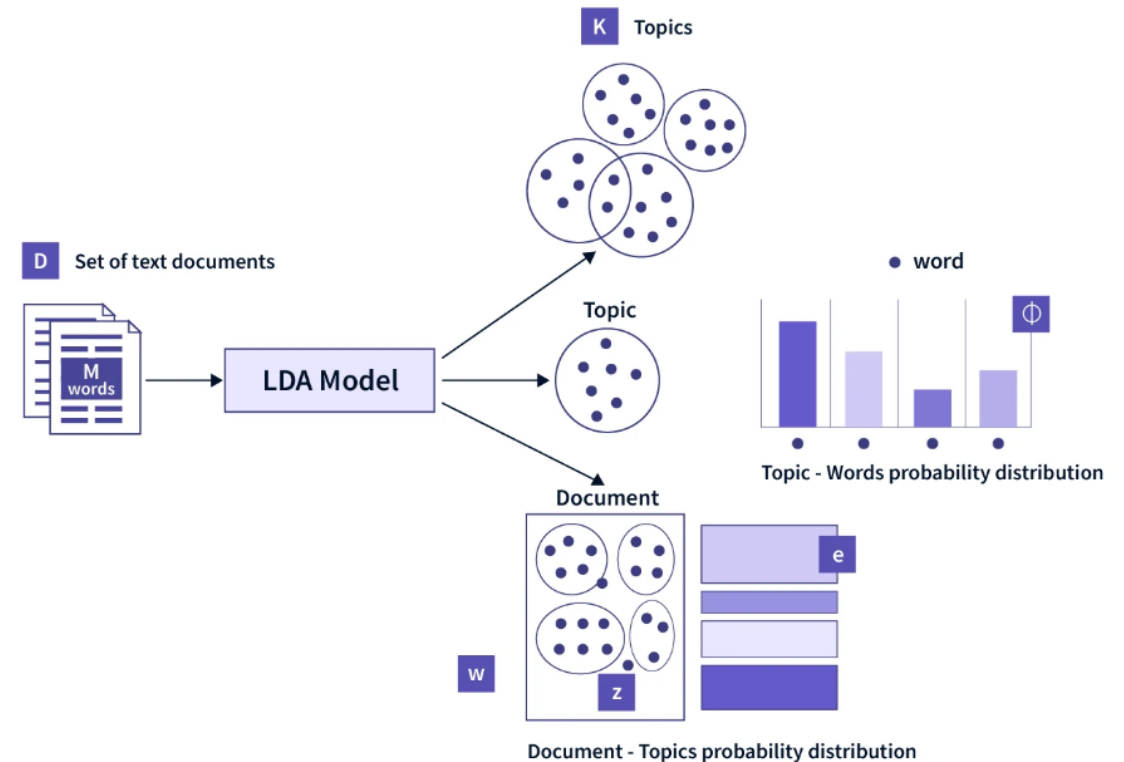
$\Sigma =$	2.285	0	0	0	0
	0	2.010	0	0	0
	0	0	1.361	0	0
	0	0	0	1.118	0
	0	0	0	0	0.797



Données textuelles

Topic modeling – Latent Dirichlet Allocation (LDA)

- Approche probabiliste non-supervisée
 - Un nombre k de « topic » (prédéfini)
 - Un topic est attribué à chaque mot du corpus en suivant une [distribution de Dirichlet](#)
 - Un document est représenté par un mix de topic
 - On cherche à améliorer l'affectation de chaque document à chaque topic en mettant itérativement à jour:
 - $p(t|d)$, la probabilité que le document d soit assigné au topic t
 - $p(w|t)$, la probabilité que le thème t soit assigné au mot w
 - Le modèle se stabilise au bout d'un certain nombre d'itération
- Souvent préféré à LSA
- Mais temps de calcul coûteux comparé à LSA



Données textuelles

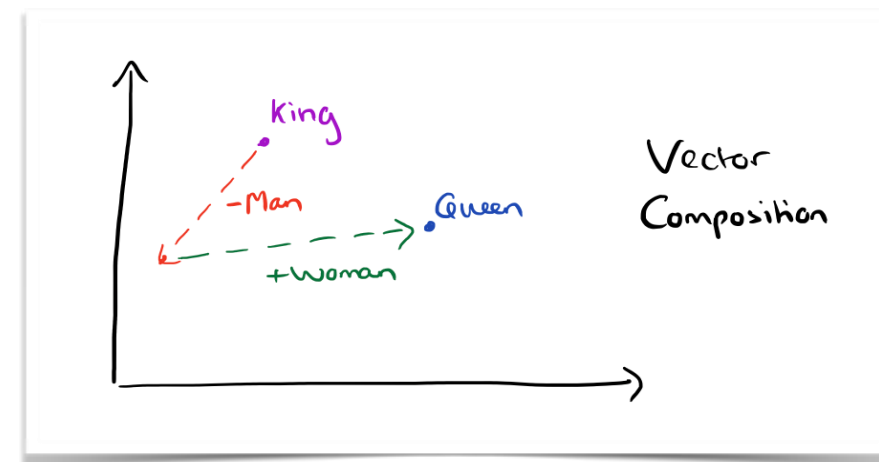
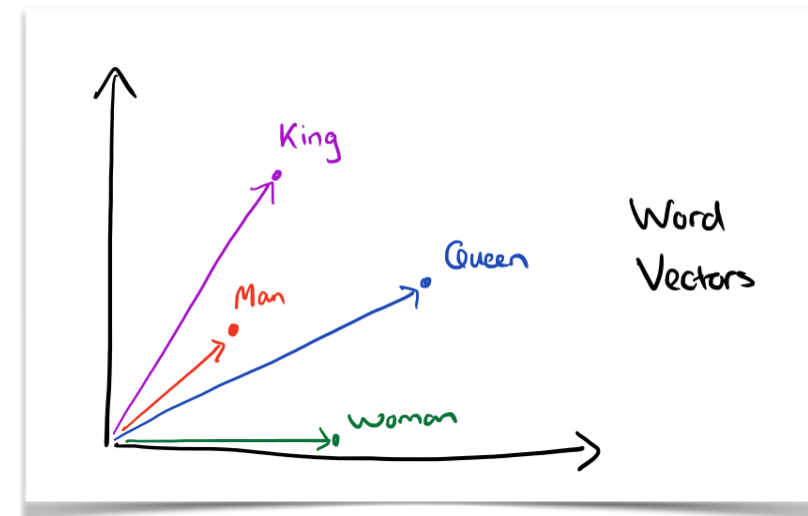
Vectorisation (*embeddings*)

	King	Queen	Woman	Princess	...
Royalty	0.99	0.99	0.02	0.98	
Masculinity	0.99	0.05	0.01	0.02	
Femininity	0.05	0.93	0.999	0.94	
Age	0.7	0.6	0.5	0.1	
...	...				

- Approches « classique »:
 - Bag of Words, TF-IDF, etc.
- Approches actuelles:
 - Deep Learning et transformers: BERT, GPT, etc.

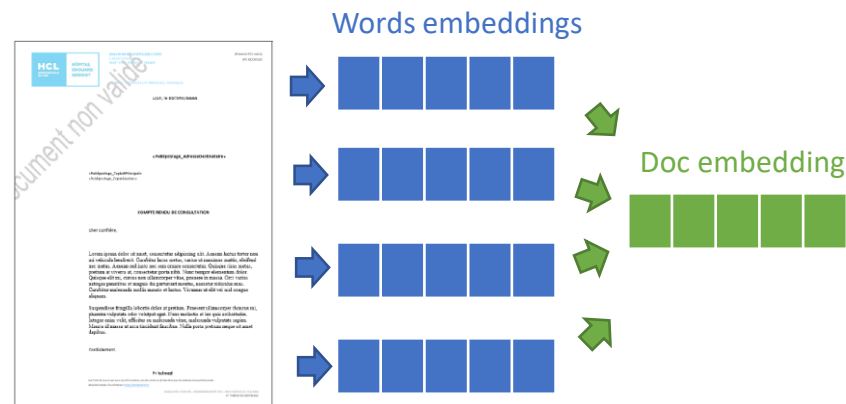
⇒ Une fois sous forme de vecteur, un texte peut être traité comme un jeu de données structuré

⇒ Donc on peut y appliquer tout ce qu'on a vu avant :D

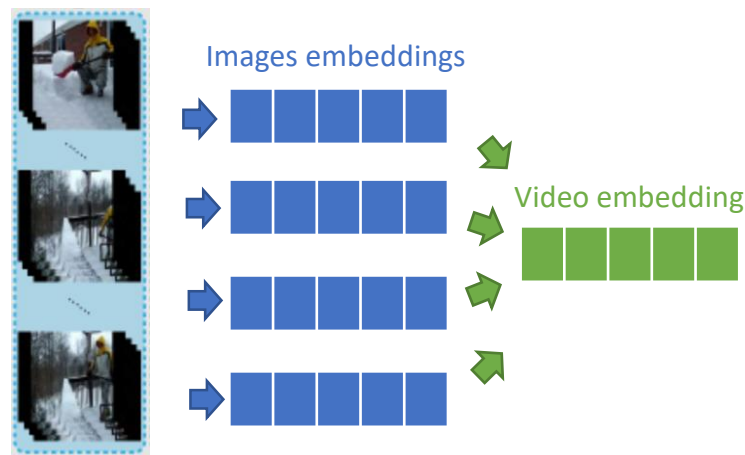


Vectorisation (*embeddings*)

Fonctionne pour tout type de données non-structurées



Doc2Vec

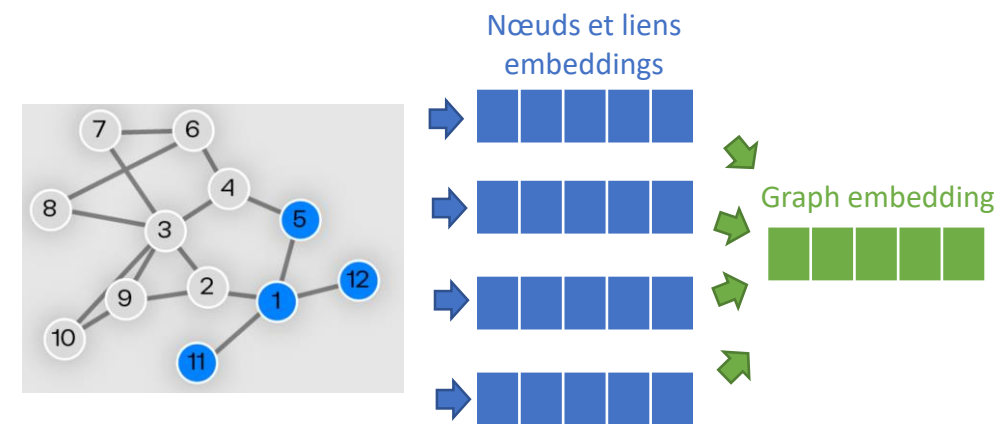


Video2Vec



Image embedding

Img2Vec



Graph2Vec

➡ Multi-modalité !

Place à l'action !



Votre Mission:

- Reprendre votre code là vous l'aviez laissé
 - Lien initial: <https://github.com/a-t-richard/20262027-UE-DataMining-Project>
- Utiliser les nouvelles notions et méthodes vues pendant ce cours
- Trouver ce qu'il se cache dans ces données !
- Rendu avant le 30/10/2026 à 23:59
 - (Le vendredi avant l'examen du 03/11)